Spam Email Detection System

PROJECT REPORT

BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING

By
Batch – 10

K. Varshini (323103210119)
K. Sai Tejaswi (323103210102)
G. Vineetha (323103210241)
K. Udaya Sri (323103210100)

Under the esteemed guidance of
Dr. N. Sharmili
Professor
CSE Department

Department of Computer Science and Engineering

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

[Approved by AICTE NEW DELHI, Affiliated to Andhra University, Visakhapatnam]
[Accredited by National Board of Accreditation (NBA) for B. Tech. CSE, ECE & IT – valid from 2019-22 and 2022-25]
[Accredited by National Assessment and Accreditation Council (NAAC) with A Grade-Valid from 2022-2027]
Kommadi, Madhurawada, Visakhapatnam – 530048

2023 – 2027

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled "**Spam Email detection**" is the bonafide work of following III B.Tech. students from the Department of Computer Science and Engineering, Gayatri Vidya Parishad College of Engineering for Women, affiliated with AndhraUniversity, Visakhapatnam. This project was done **as part of Cryptography and Network Security lab** during the academic year 2025–2026 (III Year Semester-2).

K. Varshini (323103210119)
G. Vineetha (323103210241)
K. Sai Tejaswi (323103210102)
K. Udaya Sri (323103210100)

Signature of the Guide

Dr. N. Sharmili MTech., Ph.D.
Professor,
Dept. of CSE

Signature of the HOD

Dr. P. V. S. L. Jagadamba MTech, Ph.D
Professor,
Dept. of CSE

External Examiner

ABSTRACT

Spam emails are one of the major challenges in digital communication, causing inconvenience, security risks, and loss of productivity. Detecting and filtering such unwanted messages is essential to ensure safe and efficient email usage. This project presents a **Spam Email Detection Web Application** developed using Machine Learning techniques.

The system uses the **Naive Bayes algorithm**, a probabilistic classifier, along with **TF-IDF (Term Frequency-Inverse Document Frequency)** for converting textual data into numerical form. The dataset containing spam and non-spam (ham) emails is pre-processed to remove noise, improve data quality, and enhance model performance.

A web application is developed using the **Flask** framework, which allows users to input email content and instantly receive predictions. The trained model classifies the given text as either **Spam** or **Not Spam**, providing a simple and user-friendly interface.

The proposed system is efficient, easy to use, and demonstrates how machine learning can be applied to solve real-world problems. It can be further enhanced by using advanced algorithms and integrating with real-time email services for improved accuracy and scalability.

1.SYSTEM REQUIREMENTS

1.1. Software Requirements

Table 1. Software Requirements

Software	**Version / Specification**
Python	3.x
Visual Studio Code Editor	Latest
Flask	Web Framework
Operating System	Windows/Linux
Scikit-learn	ML Library
NumPy	Numerical Computing
Pandas	Data Handling

Python packages

Python packages we used in our project includes the following table 2.

We used the command pip install "*package_name==version*" to download the packages.

Table 2. Python packages

S.No.	**Package Used**	**Description**
1.	NumPy	Numerical operations
2.	Pandas	Data processing
3.	Scikit-learn	Machine learning
4.	Pickle	Model saving
5.	Matplotlib	Visualization
6.	Seaborn	Data visualization
7.	Flask	Web framework
8.	Regex (re)	Text cleaning

Front End Packages used

Table 3. Front end Packages

S.NO	Libraries Used	Description
1.	HTML	Web page structure
2.	Jinja2	Template engine
3.	CSS	Styling and layout design
4.	JavaScript	Used to add interactivity and improve user experience

3.3.2 Hardware Requirements

The below Table 4 shows the minimum hardware requirements of our project.

Minimum Hardware Requirements:

Table 4. Minimum Hardware Requirements

S.No	Hardware Component	Specification
1.	Processor	Intel i3 or above
2.	RAM	4 GB minimum
3.	Hard Disk	500 GB
4.	System Type	64-bit
5.	Internet	Required

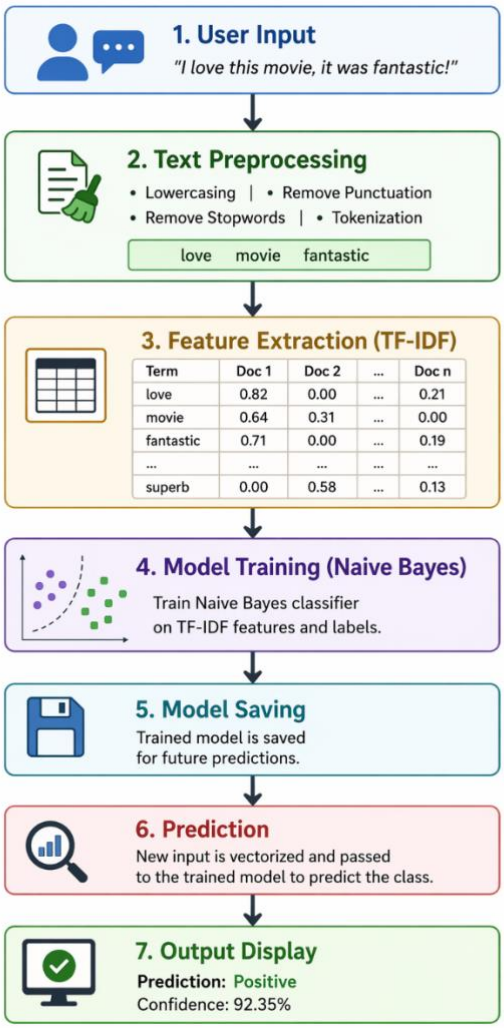
2.METHODOLOGY

2.1 SYSTEM ARCHITECTURE

The proposed system follows a machine learning pipeline to classify emails as spam or not spam. The architecture consists of multiple stages starting from user input to final prediction.

Flow:

- User provides email text through web interface
- Text is cleaned and processed
- Features are extracted using **TF-IDF**
- Classification is done using **Multinomial Naive Bayes**
- Result is displayed on the webpage



2.2DESCRPTION, algorithm

The Spam Email Detection System works by processing email text and applying machine learning techniques to classify messages as spam or not spam. The system follows a step-by-step approach as described below:

Step 1: Data Collection

The system uses a dataset containing email messages labeled as **spam** and **ham (not spam)**.

This dataset is used to train the model so that it can learn the patterns of unwanted and legitimate emails. A good dataset improves the accuracy of the system.

Step2: Data Preprocessing

The collected data is cleaned and prepared before training the model

This includes:

- Converting all text into lowercase to maintain uniformity
- Removing punctuation marks and special symbols
- Eliminating unwanted or irrelevant words
- Preprocessing helps in reducing noise in the data and improves the performance of the model.

Step 3: Feature Extraction

Since machine learning models cannot understand text directly, the text is converted into numerical format using

TF-IDF (Term Frequency-Inverse Document Frequency).

- It assigns importance to words based on their frequency
- Words that appear frequently in a specific email but rarely in others get higher importance
- This step transforms text into vectors that can be processed by the model.

Step 4: Model Training (Algorithm)

The processed data is used to train a classification model using **Multinomial Naive Bayes**.

- It is a probabilistic algorithm based on Bayes' theorem
- It calculates the probability of an email being spam or not spam based on word occurrences
- The model learns from the training data and builds a prediction model
-

Step 5: Prediction

Once the model is trained, it is used to classify new email messages.

- The user inputs a message into the system
- The same preprocessing and feature extraction steps are applied
- The trained model predicts the category

  The output is displayed as:
- **Spam Email**
- **Not Spam**

3. IMPLEMENTATION

Source code:

App.py

```
from flask import Flask, render_template, request
import pickle
app = Flask(__name__)
# Load model and vectorizer
model = pickle.load(open("spam_model.pkl", "rb"))
vectorizer = pickle.load(open("vectorizer.pkl", "rb"))
@app.route('/')
def home():
    return render_template("index.html")
@app.route('/predict', methods=['POST'])
def predict():
    message = request.form['message']
    data = vectorizer.transform([message])
    prediction = model.predict(data)
    if prediction[0] == 1:
        result = "Spam Email 🚫"
    else:
        result = "Not Spam ✅"
    return render_template("index.html", prediction=result)
if __name__ == "__main__":
    app.run(debug=True)
```

model.py:

```
import pandas as pd
import string
import pickle
import os
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
# Check if file exists
if not os.path.exists("spam.csv"):
    print("ERROR: spam.csv file not found in this folder!")
    exit()
# Load dataset safely
try:
    data = pd.read_csv("spam.csv", encoding="latin-1")
except Exception as e:
    print("Error reading CSV:", e)
    exit()
# Check if file is empty
if data.empty:
    print("ERROR: spam.csv file is empty!")
    exit()
print("Dataset Loaded Successfully!")
```

```
# Keep required columns safely
if 'v1' in data.columns and 'v2' in data.columns:
    data = data[['v1', 'v2']]
else:
    print("ERROR: Required columns not found in dataset!")
    print("Columns found:", data.columns)
    exit()
data.columns = ['label', 'message']
# Convert labels
data['label'] = data['label'].map({'ham': 0, 'spam': 1})
# Clean text
def clean_text(text):
    text = str(text).lower()
    text = ''.join([char for char in text if char not in string.punctuation])
    return text
data['message'] = data['message'].apply(clean_text)
# Vectorization
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(data['message'])
y = data['label']
# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Train Model
model = MultinomialNB()
```

```
model.fit(X_train, y_train)
# Accuracy
pred = model.predict(X_test)
print("Model Accuracy:", accuracy_score(y_test, pred))
# Save files
pickle.dump(model, open("spam_model.pkl", "wb"))
pickle.dump(vectorizer, open("vectorizer.pkl", "wb"))
print("Model Saved Successfully!")
```

spam.CSV:

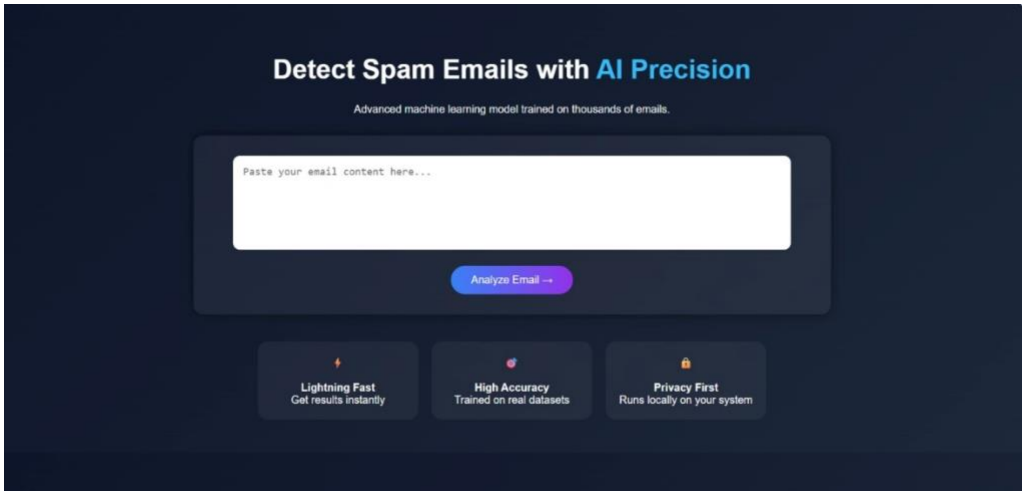
```
v1,v2
ham,Hey how are you
spam,Congratulations! You won 1 crore rupees
ham,Are you coming to class tomorrow?
spam,Claim your free prize now!!!
ham,Let's meet at 5 pm
spam,Click this link to get free recharge
ham,Call me when you are free
spam,Limited offer! Buy now!
ham,Did you finish the assignment?
spam,You have been selected for lottery
```

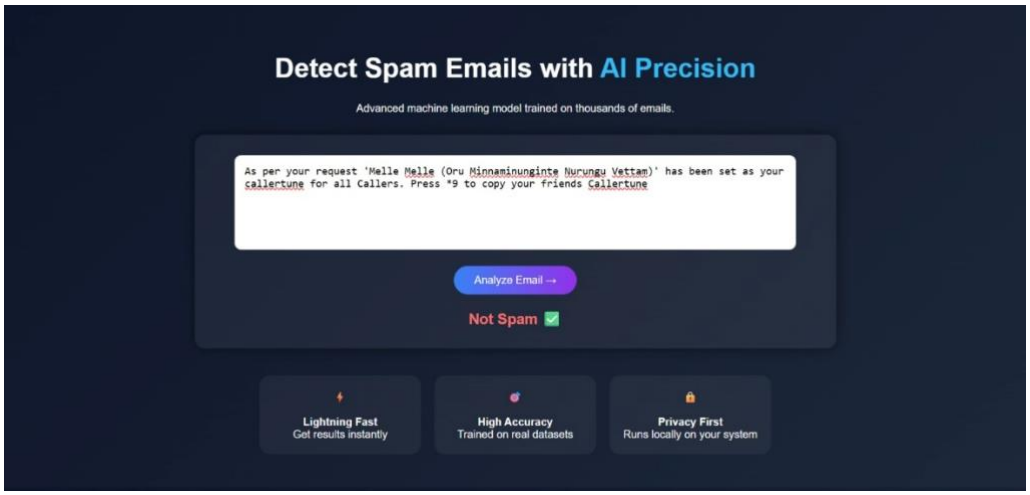
index.html:

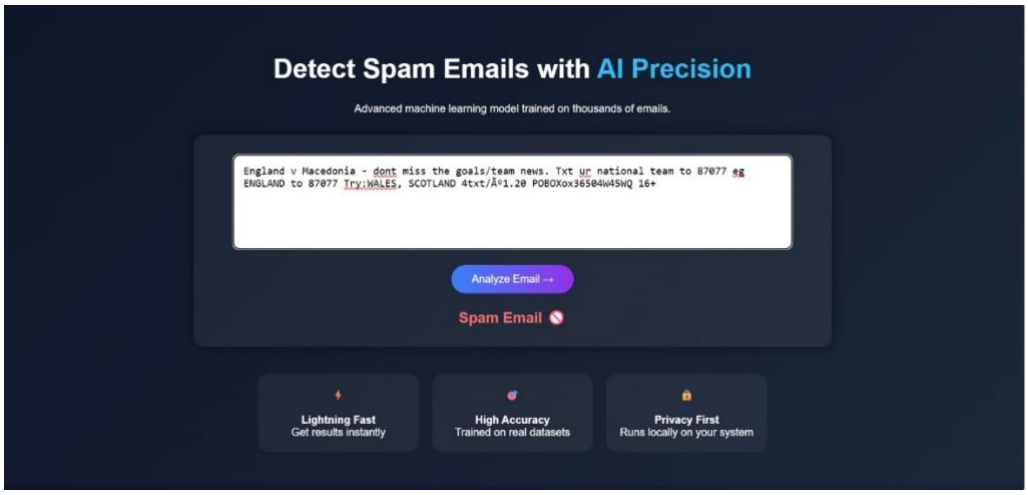
```
<!DOCTYPE html>
<html>
<head>
  <title>Email Spam Detection</title>
</head>
<body style="text-align:center; margin-top:50px;">
  <h2>Email Spam Detection System</h2>

  <form action="/predict" method="POST">
    <textarea name="message" rows="6" cols="50" placeholder="Enter
your email text here..."></textarea>
    <br><br>
    <button type="submit">Check</button>
  </form>

  {% if prediction %}
    <h3>{{ prediction }}</h3>
  {% endif %}
</body>
</html>
```

4. RESULTS

7.1 Output Screens







5.CONCLUSION

The Spam Email Detection System developed in this project effectively classifies email messages into spam and non-spam categories using machine learning techniques. The implementation of **Multinomial Naive Bayes** along with **TF-IDF (Term Frequency-Inverse Document Frequency)** plays a crucial role in achieving accurate and efficient predictions.

The system successfully preprocesses textual data, extracts meaningful features, and applies a trained model to identify unwanted messages. It provides a simple and user-friendly interface through a web application, allowing users to easily check whether an email is spam or not.

This project demonstrates the practical application of machine learning in solving real-world problems such as email filtering and cybersecurity. It also highlights how text classification techniques can be effectively used to improve communication systems and reduce the risk of spam-related threats.

Overall, the system is efficient, reliable, and serves as a strong foundation for further improvements and real-time deployment.

6.FUTURE SCOPE

Although the current system performs well, there are several ways to enhance its functionality and accuracy in the future:

- **Use of Deep Learning Techniques**:
  Advanced models such as neural networks and recurrent neural networks can be used to improve classification accuracy and handle complex patterns in text data.
- **Larger and More Diverse Datasets:**
  Training the model with a larger dataset containing different types of spam emails can improve generalization and reduce misclassification.
- **Integration with Email Services:**
  The system can be integrated with platforms like Gmail or Outlook to automatically filter spam emails in real time.
- **Real-Time Deployment**:
  Deploying the system on cloud platforms will allow users to access the application from anywhere and enable scalable usage.
- **Multi-Language Support:**
  Extending the system to detect spam in different languages can make it more versatile and widely usable.
- **Improved User Interface:**
  Enhancing the frontend design can provide a better user experience and interaction.

7.REFERENCES

The following resources were used for developing this project:

1. Official documentation of **Scikit-learn**
    - Provides tools and algorithms for machine learning implementation.
2. Official documentation of **Python**
    - Used for coding and implementing the system.
3. Tutorials and guides on Machine Learning concepts
    - Helped in understanding classification algorithms and text processing.
4. Research papers and online resources on spam detection techniques
    - Provided insights into different approaches and improvements.
5. Documentation for **Flask**
    - Used for developing the web application interface.